EF Core Change Tracking

Lesson Summary • Core concepts, entity states, and SaveChanges()

1. What Is Change Tracking?

Change Tracking means EF Core watches tracked entity objects and remembers changes so it can save them later. It can compare original and current values, detect modified properties, and determine what SQL is needed.

2. The Core Workflow

1. Query Load entity	**2. Track** DbContext	**3. Modify** C# property	**4. Detect** Changes	**5. Save** SQL sent

3. Practical Example

```
var student = context.Students
    .First(s => s.StudentId == 1);

student.Status = "Inactive";

context.SaveChanges();
```

Changing **Status** changes the in-memory object. Calling **SaveChanges()** asks EF Core to persist the change; it may generate an UPDATE statement for that row.

4. What EF Core Can Track

• Entity identity and original values

• Current values and modified properties

• Added and deleted entities

• Relationship changes

5. Entity States

State	Meaning
Detached	Not tracked by the context.
Unchanged	Tracked; no changes detected.
Modified	One or more properties changed.
Added	New entity marked for insertion.
Deleted	Entity marked for deletion.

6. State Flow

Query → **Unchanged** → edit property → **Modified** → **SaveChanges()** succeeds → **Unchanged** again.

7. AsNoTracking() and Performance

Tracking has a cost in **memory**, **CPU**, and internal bookkeeping. For read-only queries, **AsNoTracking()** can avoid tracking overhead. The lesson introduces this idea; detailed behavior comes later.

Common mistake: Changing a property does **not** update the database immediately. It changes the object in memory. Database persistence happens when **SaveChanges()** is called.

8. Mental Model

EF Core tracks objects first. SQL happens later on SaveChanges(). Change Tracking supports automatic updates, relationship synchronization, batch saves, and unit-of-work workflows.